

Code Review — createStudents.js

Overview

When I read `createStudents.js`, I imagined the real situation it has to handle. A teacher wants to upload her class — maybe an Excel file with hundreds, maybe thousands of students — into the system. Every student needs a unique username within that school. Before anything is saved, the teacher wants to know whether the file is clean: are all the usernames unique, do all the fields validate? If there's a problem, she should see what's wrong and fix it. Only when the preflight check passes does the real save happen.

The code does this. The author has clearly thought about it — atomic batches with transactions, per-row error collection with savepoints, a preflight mode that doesn't persist passwords during a dry run. The implementation works.

What I found is mostly about scaling and structure, not correctness. The code does more work than it needs to, and it does it in the wrong order — many of the checks happen inside the database transaction when they could happen in JavaScript first. Things like duplicate usernames in the same batch, empty fields, missing required values — none of these need a database connection to detect. The biggest example is the per-row INSERT loop — for 10,000 students it makes over 20,000 round trips to Postgres. Section 5 walks through this with measurements: my hybrid replacement (pre-validation in JavaScript + one SELECT for existing usernames + one bulkCreate) handles 10,000 students in 782ms instead of an estimated 20 seconds.

For context: I built a parallel Express/Sequelize/Postgres project from scratch to hit the same problems this file solves, then iterated through several refactors of my own version. Verification of the suggestions in this review is captured in `script/batch-test.js` and `test-results.txt`.

Strengths

A few things in this code are genuinely well thought through, and I want to flag them before getting into the issues:

- **Savepoint-based per-row error collection.** Using `SAVEPOINT` inside a transaction to collect every row's error in one pass — rather than failing on the first bad row — is a smart use of Postgres. The user gets every problem back in one response instead of fixing them one at a time across multiple retries.
- **discardPassword: preflight option.** Wiring this into the model so passwords aren't persisted during a dry-run is a small but real security win. Preflight gets called often, and avoiding password exposure in transaction memory and DB logs on every call adds up.
- **Composite-unique noise filter.** Filtering out the `schoolId / not_unique` error when a composite unique index is violated is defensive code that recognises a real Sequelize quirk. Most developers would only notice this after hitting it in testing.
- **Clean preflight design.** Same code path for verified and created — only the final step differs (rollback vs commit). The whole pipeline runs both ways, so preflight gives you a faithful prediction of what the real save would do.

- **Separation of concerns.** Validators on the model. Formatting in `formatValidationErrors`. Error reporting in `reportError`. The controller stays focused on control flow.

Tools used

- **Editor and runtime:** VS Code, Node.js v22, Postgres 17 in Docker
- **Libraries:** Express, Sequelize v6, dotenv
- **DB inspection:** Beekeeper Studio
- **AI assistance:** I used Claude (Anthropic) throughout this project as a pair-programmer and tutor. I drafted the initial loop-with-savepoints implementation by hand, then worked through refactors one at a time — verifying each one with the batch test script before moving on. The review prose was drafted in conversation with Claude and edited in my own voice. My broader approach to AI-assisted learning is documented in `LEARNING_STYLE.md`.
- **Verification:** A batch test script (`script/batch-test.js`) exercises 10 scenarios — school-not-found, empty batch, oversize batch, in-batch duplicates, field validation, pre-existence conflicts, preflight, real save, and others. Output is in `test-results.txt`. For scaling claims in section 5, I ran additional load tests at 10,000, 85,000, and 1,000,000 students against my parallel implementation; the timings and failure modes are cited in that section.

Issues

1. Configuration handling: maxStudents

Analyse

```
const maxStudents = process.env.MAX_STUDENTS_PER_REQUEST || 50
```

This is read inside the handler on every request.

Problems

- **Function scope.** It's currently reading the environment variable inside the function. There's no reason to read it on every request — the env doesn't change while the process is running. It should be at module scope so it runs once when Node starts and lives in memory.
- **String vs number.** Environment variables always return strings. We need to parse it to a number and check the result is valid. Otherwise the comparison with `req.body.length` later in the code can give the wrong result.
- **The `|| 50` killswitch trap.** The killswitch behaviour itself is good practice — being able to disable a risky feature via config without a redeploy is exactly what you want in production. But the current implementation works by accident, because `"0"` is a non-empty string and therefore truthy. The first developer who tries to tidy the type handling — say, by writing `parseInt(...)` `|| 50` — will silently break the killswitch, because `0 || 50` returns 50.

Improvements

Move it to module scope, parse it explicitly, and guard against `NaN`:

```
const fromEnv = parseInt(process.env.MAX_STUDENTS_PER_REQUEST, 10)
const MAX_STUDENTS = Number.isNaN(fromEnv) ? 50 : fromEnv
```

Two lines, no operator tricks. Handles missing env, invalid input, and the 0 killswitch consistently — `MAX_STUDENTS` is always a number.

Feedback

Use `ALL_CAPS` for module constants. The commented-out alternative in the original should be removed.

2. In-batch duplicate detection: $O(N^2)$ and inside the transaction

Analyse

For each student in the batch, the original checks for duplicates by filtering the entire request body:

```
for (let [index, student] of req.body.entries()) {
  // ...
  const duplicatedUsername = req.body.filter((s) => s.username ===
student.username).length > 1
}
```

This check runs **inside the transaction**, after `Database.transaction()` has already been opened.

Problems

- **$O(N^2)$ complexity.** `filter()` walks the entire batch for every iteration of the outer loop. For 50 students that's 2,500 comparisons. For 10,000 students it's 100 million.
- **Runs after the transaction is open.** If the batch has duplicates, a transaction has already been acquired and savepoints opened — all of which has to be rolled back. The duplicate check is pure JavaScript. It doesn't need a database connection at all.

Improvements

Use a `Set` to detect duplicates in a single pass, before opening the transaction:

```
const usernameTracker = new Set()
const duplicateUsernames = new Set()

for (const student of req.body) {
  if (usernameTracker.has(student.username)) {
    duplicateUsernames.add(student.username)
  } else {
    usernameTracker.add(student.username)
  }
}

if (duplicateUsernames.size > 0) {
```

```
// build error array, respond 400, return – no transaction needed
}
```

One pass. `Set.has()` and `Set.add()` are $O(1)$. The function fails fast when the batch is malformed, and no database work happens for batches that were never going to succeed.

Feedback

Cheap JavaScript validation should happen before acquiring database resources. The original works correctly but scales badly. The fix also simplifies the inner loop — the duplicate-check block disappears from the per-student try/catch.

3. Field-level validation runs inside the transaction

Analyse

The original calls `Student.create(...)` inside the per-row loop. Sequelize runs the model's column validators (`notEmpty`, type checks, `allowNull`) as part of `create`. If a row has an empty name or a missing field, the savepoint rolls back and the error is recorded.

```
await Database.query('SAVEPOINT saved', { transaction })
const createdStudent = await Student.create(rowData, { transaction })
// validation errors caught in catch, savepoint rolled back
```

Problems

- **Validation happens after the transaction is open.** A row with an empty name forces an open transaction, a savepoint, an INSERT attempt, and a rollback — none of which is needed to detect an empty string in JavaScript.
- **Pure-JS errors share the same flow as DB-only errors.** Field-level checks (which only need JS) and concurrency-driven errors (which only the DB can know) get caught in the same place. That makes the flow harder to reason about.

Improvements

Use `Student.build(data).validate()` to run the model's validators **without** touching the database, before opening any transaction:

```
const validationErrors = []
for (const [index, studentData] of req.body.entries()) {
  try {
    await Student.build({ ...studentData, schoolId, createdBy: req.user.id })
      .validate()
  } catch (err) {
    if (err instanceof Sequelize.ValidationError) {
      for (const e of err.errors) {
        validationErrors.push({
          path: `${index}.${e.path}`,

```

```

        errorCode: e.validatorKey,
        message: e.message,
        location: 'body',
      })
    }
  }
}

if (validationErrors.length > 0) {
  res.status(400).json({ errors: validationErrors })
  return
}
// only now open the transaction

```

`build()` creates an unsaved instance. `validate()` runs the same column validators `create()` would, without sending an INSERT. Errors are collected in JavaScript, with row indices, before any DB resources are acquired.

Feedback

This separates JavaScript concerns (data shape) from database concerns (concurrent state). What pre-validation catches: column validators, type checks, `allowNull`. What it can't catch — and what still has to live at the DB level — is anything the database alone knows about: unique constraint violations from race conditions, foreign key violations if the school is deleted mid-request. So the savepoint pattern inside the loop still has a role, but only for those genuine DB-only errors.

4. No shape validation on `req.body`

Analyse

The original assumes `req.body` is an array of student objects. There's no check that it's actually an array, or that it contains anything.

Problems

- A client sending `null`, an object, or a string would crash the handler on `.length` or `.filter`.
- An empty batch `[]` succeeds with no inserts, but still opens a transaction.

Improvements

A quick fix is two manual checks at the top of the handler:

```

if (!Array.isArray(req.body)) {
  res.status(400).json({ errors: [...] })
  return
}
if (req.body.length === 0) {
  res.status(400).json({ errors: [...] })
}

```

```
    return  
  }
```

A better fix is a schema library like Zod or Joi — you declare the expected shape once, and it validates the whole body in one pass:

```
const BodySchema = z  
  .array(  
    z.object({  
      name: z.string().min(1),  
      username: z.string().min(1),  
      password: z.string().min(1),  
    }),  
  )  
  .min(1)  
  .max(50)
```

One source of truth for the API contract, with detailed errors for free.

Feedback

Shape validation should be the first thing the handler does. It guards every line below it from assumptions about the body's structure.

5. Scaling: per-row INSERTs vs bulk operations

Analyse

The original processes students in a loop, calling `Student.create` once per row inside a transaction. For each student that's three SQL statements — a `SAVEPOINT`, the `INSERT`, and sometimes a `ROLLBACK` if validation fails. All of it serial, all inside one open transaction.

Problems

This is **$O(N)$ database round trips**. Three things compound at scale:

- **Latency** — N round trips dominate the response time.
- **Lock contention** — the transaction holds for the whole duration, blocking other writes to the table.
- **Timeouts** — at 30+ seconds the HTTP layer times out, even though the work is still happening on the server.

The loop pattern only makes sense when per-row error reporting is genuinely needed *and* pre-validation isn't possible.

Improvements

I implemented and tested the alternative end to end:

1. **Pre-validate every row in JavaScript** with `Student.build(data).validate()` — catches field errors with row indices, no DB.

2. Pre-check existing usernames in one query:

```
const existing = await Student.findAll({
  where: {
    schoolId: req.params.schoolId,
    username: { [Op.in]: req.body.map((s) => s.username) },
  },
  attributes: ['username'],
})
```

Conflicts are mapped back to row indices using a `Set`, so the error response still has full per-row attribution.

3. `bulkCreate` the clean rows in one round trip:

```
await Student.bulkCreate(records, { transaction, validate: true, returning:
true })
```

That's three round trips total, regardless of batch size: pre-existence SELECT, bulk INSERT, COMMIT.

A race is still possible — another request could insert a conflicting username between the pre-check and the bulk insert. The hybrid wraps `bulkCreate` in a try/catch and returns a 409 with the same error shape if `UniqueConstraintError` fires.

Measured results

I tested this against a local Postgres in Docker:

Batch size	Result	Notes
10,000 students (clean)	201 in 782ms	One pre-check SELECT, one <code>bulkCreate</code> INSERT, one COMMIT
10,000 students (all conflict)	400 in 231ms	Pre-check returned every conflict with row index; no transaction opened
85,000 students	Single INSERT	One <code>INSERT ... VALUES (...) RETURNING ...</code> with 85k rows, verified in the SQL log
1,000,000 students	Server crashed	FATAL ERROR: Reached heap limit — Node process ran out of memory

The 85k result was a surprise — I expected to hit Postgres's parameter limit (~65,535) much earlier, but empirically Sequelize and Postgres handled it in one statement.

A safer next step — chunked `bulkCreate`

Sending all 10,000 rows in one `bulkCreate` worked in my testing, but production safety is better served by chunking the insert. Splitting the clean rows into batches of, say, 1,000 and looping `bulkCreate` over them keeps each INSERT small — well under any parameter or memory limit:

```
const CHUNK_SIZE = 1000
const transaction = await Database.transaction()
try {
  for (let i = 0; i < records.length; i += CHUNK_SIZE) {
    const chunk = records.slice(i, i + CHUNK_SIZE)
    await Student.bulkCreate(chunk, { transaction, validate: true,
returning: true })
  }
  await transaction.commit()
} catch (err) {
  await transaction.rollback()
  throw err
}
```

The transaction still wraps the whole batch, so atomicity is preserved — every chunk commits, or none of them does. No savepoints needed: if any chunk throws, the catch block rolls everything back. The win is that no single INSERT carries the whole batch, which makes the failure mode more predictable.

Beyond the hybrid — async job processing

The 1M test exposed the real failure mode of the synchronous pattern. It isn't slow responses, it's the **Node process running out of memory and crashing**. Every in-flight request dies with it. No amount of tuning fixes that — the request/response pattern itself is wrong for genuinely large imports.

For production scale, the answer is **async job processing**. The HTTP endpoint validates the request, puts it on a queue (BullMQ on Redis, or pg-boss on Postgres), and returns immediately with a job ID. A separate worker consumes the queue, processes chunks of 500–1,000 rows in independent transactions, and tracks progress. The frontend polls or subscribes via websocket; failed rows are downloadable as a CSV. This is the pattern Stripe, Mailchimp, and Ghost use.

A simpler intermediate step is **client-side chunking** — the frontend splits a 10,000-row upload into 5 batches of 2,000. Server stays under default body limits and each batch is atomic. The trade-off is no cross-batch atomicity — if batch 3 of 5 fails, batches 1–2 are already committed. For a school admin workflow, where the user can just re-upload the failed rows, this is usually fine.

Body parser note

The default Express body parser rejects payloads over 100KB with a **413 Payload Too Large** HTML response (not JSON, which breaks the API's error shape). Raising the limit isn't a real fix — it just moves the failure from the HTTP layer to memory exhaustion in the handler. For genuinely large imports, the answer is the async job pattern, not a bigger body limit.

Feedback

The savepoint-per-row pattern is over-engineered for current scale and under-engineered for future scale. The hybrid (pre-validation + pre-existence check + bulkCreate) gives you the same per-row error attribution with three round trips total — comfortable up to ~10,000 students. Chunked bulkCreate is the safer next step within the same request/response shape. Beyond that, the architecture itself needs to change: async job processing for genuinely large imports, or client-side chunking as a simpler intermediate.

6. Response shape inconsistency

Analyse

The original returns errors in two different shapes depending on the failure:

- School-not-found and too-many-students return **a bare array**:

```
res.status(404).json([ { path: 'schoolId', errorCode: 'ERR_SCHOOL_NOT_FOUND', ... } ])
```

- Validation errors return **an object with an errors key**:

```
res.status(400).json({ errors: [...] })
```

Success responses use yet another shape: { **verified**: [...] } or { **created**: [...] }.

Problems

The frontend has to branch on the response shape to know how to parse errors. **response.errors** works for some failures, **response[0]** for others. Easy to get wrong, and surprising for anyone expecting a single error envelope across the API.

Improvements

Standardise on the object form for every error response:

```
res.status(404).json({ errors: [ { path: 'schoolId', errorCode: 'ERR_SCHOOL_NOT_FOUND', ... } ] })
```

Now the frontend can write one error-rendering routine that reads **response.errors**, no matter which check fired. It also leaves room to add metadata later — a **meta** field for request IDs, pagination on the success side — without breaking consumers.

Feedback

API consistency is a force multiplier. The frontend gets one contract to rely on, and new endpoints don't need new parsing code. Worth fixing across the board.

7. Outer catch returns 500 cleanly — but watch for the easy mistake

Analyse

The outer catch in the original is correct:

```
} catch (error) {  
  reportError(error)  
  res.status(500).send()  
}
```

I'm flagging it because the wrong version of this is easy to write, and worth calling out:

```
} catch (error) {  
  res.status(400).json({ error: error.message })  
}
```

Problems with the bad pattern

- **Wrong status.** Unexpected errors in the outer catch are server bugs, not client mistakes. A 4xx tells the client "you did something wrong"; a 5xx tells them "the server broke." Status codes are how clients and monitoring decide what to do.
- **Information leak.** Raw `error.message` from Postgres or Sequelize can contain table names, SQL fragments, file paths, or stack-trace bits. None of that should reach a client.

The original gets this right — log via `reportError`, return an empty 500. The client knows the server broke; the developer sees the full error in logs.

Feedback

Treat the outer catch as the "unexpected error" branch — log server-side, return a generic 500. The inner per-row catches handle *expected* failures (validation, constraints) with detail and a 4xx. The two should look visibly different in code so the intent is obvious.

8. Sequence ID burning on preflight rollback

Analyse

Preflight runs the full pipeline — including the `INSERT`s — then rolls back the transaction. The inserted rows are gone, but **the Postgres sequence that allocates primary keys is non-transactional and does not roll back**. Every preflight call permanently consumes IDs.

I verified this in testing: a preflight call returned `verified: [1, 2]`, and the following real save returned `created: [3, 4]`. The IDs 1 and 2 were burned by the rolled-back preflight.

Problems

If preflight is called frequently — e.g., a UI that previews on every form change — the gap between *student count* and *highest student ID* will grow without limit. With a 4-byte INTEGER sequence the ceiling is ~2.1 billion, which is generally fine, but:

- IDs in audit logs become harder to map to real rows.
- Seeing "student #84,203" when there are only 80,000 students is confusing.
- For tables using smaller integer types, exhaustion is a real risk.

This isn't a bug in Postgres or Sequelize. Sequences are deliberately outside transactions, so they don't serialise across concurrent writers. It's a trade-off — and the cost lands on any rollback-based preflight.

Improvements

Two options:

1. **Pure-validation preflight.** Replace "insert then rollback" with the JavaScript pre-validation pass from section 3 — `Student.build(data).validate()` plus a `findAll` to check uniqueness. No INSERTs in preflight, so no IDs consumed.
2. **Accept the cost.** For low-volume preflight traffic, sequence-burning is harmless. Document it and move on. `BIGSERIAL` instead of `SERIAL` makes the ceiling effectively unbounded.

Feedback

Worth flagging because most people assume "rollback undoes everything." It doesn't — sequences sit outside transactional semantics on purpose. Whether to fix or accept depends on how often preflight gets called.

9. `Array.from(req.body.entries()).length` smell

The original computes the batch size with `Array.from(req.body.entries()).length`. This is the same as `req.body.length` — `.entries()` creates an iterator, `Array.from` turns it into an array, and `.length` reads its size. Three operations to get a value the array already has.

The clue is later in the same file: `for (let [index, student] of req.body.entries())` actually needs `.entries()` to get both index and value. The `Array.from(...).length` line looks like a copy-paste from there. `req.body.length` is direct, reads cleanly.

10. "Create then throw" pattern for duplicates

When the in-batch duplicate check fires, the original still calls `Student.create(...)` for the duplicate row — and then deliberately throws a fake `Sequelize.ValidationError` to force the savepoint to roll back.

The author was trying to be clever: both kinds of duplicate error (in-batch and DB-unique) end up in the same catch block, in the same shape. One error path, easy to handle. But it has two real costs:

- **A wasted INSERT round trip per duplicate.** We already know in JavaScript that the row is a duplicate, but we send it to Postgres anyway just so the catch block fires.
- **Confusing control flow.** Reading it, you see `create` succeed, then a manual throw — and you have to stop and ask "why is it throwing right after a successful create?". Code that needs a comment to make sense is usually code worth simplifying.

The fix is to detect duplicates up front in JavaScript (see section 2), and skip `create` entirely for those rows. No INSERT, no fake throw, no savepoint needed. Cleaner, faster, and the intent is obvious from the code.

11. `req.user.id` relies on middleware that isn't visible from this file

The handler reads `req.user.id` without checking that `req.user` exists. I'm guessing there's auth middleware upstream that populates it — but that dependency isn't visible from this file alone. If the route is

ever wired up without the middleware (a refactor, a test setup), the handler crashes with `Cannot read properties of undefined`.

A guard inside the handler — `if (!req.user?.id) { ... return 401 }` — or an explicit middleware requirement at the route definition would make the contract clearer. At a minimum, a comment near the line would help the next reader.

12. No migration strategy visible

The codebase uses `sequelize.sync()` to create tables on startup. That works for development and demos, but `sync()` only creates tables that don't exist — it doesn't alter existing ones. Once production data exists, every schema change (a new column, an index, a constraint) becomes a manual operation.

A proper migrations toolchain (Sequelize CLI, Umzug, or Knex) would make schema changes versioned and repeatable. Worth raising even though it's strictly outside this file — it's a deployment risk that gets harder to fix the longer it's left.